GEO-1004 Assignment-2: Enriching the 3DBAG

Ming-Chieh Hu (6186416), Neelabh Singh (6052045) & Daan Schlosser (5726042)

May 28, 2025

1 Methodology

1.1 Determining each building's volume

To determine a building's volume, we split the task into three major parts for each `buildingPart` (of LoD2.2): **constructing the mesh**, **triangulating the mesh**, and **calculating the volume from the mesh**.

1.1.1 Constructing a mesh (without holes)

To construct a mesh without holes, we defined a function `bld_mesh_from_json()` which takes a JSON object and the name of the CityObject of interest as input to extract that object's geometry as a mesh.

To constuct a mesh, we need to add vertices and specify its faces based on the indices stored in the mesh object; these are different indices from those used in the JSON file. If we naively do this by adding each face's vertices, this will result in duplicate vertices, since each vertex is connected to 3 faces. `CGAL::Surface_mesh` doesn't check for duplicated vertices, nor does it have a method to remove them. To prevent duplicates, we use an `unordered_map` to keep track of the JSON indices and the corresponding mesh indices; if we have seen this index before, we don't create a new vertex in the mesh object. We chose to use `unordered_map` because its `find()` method is on average $O(1)$ time complexity.

1.1.2 Constructing a mesh (with holes)

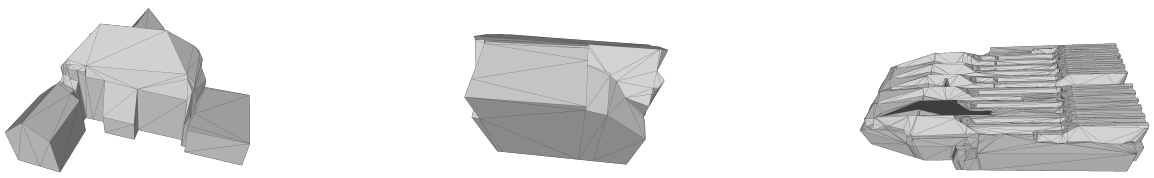
The connectivity in `CGAL::Surface_mesh` does not allow us to represent faces with holes. However, since buildings may have faces with holes, we have to find a workaround for these cases. Here, we use Triangulation_2/polygon_triangulation.cpp, from CGAL's docs, as reference for our steps:

1. For each face, define a local coordinate reference system and project 3D points to 2D.
2. Create a map `point_lift_map` to link 2D points to their 3D point indices in the mesh.
3. Extract all rings (inner and outer) and store them as polygons.
4. Create a constrained (non-Delaunay) triangulation using the polygons as constraints.
5. Iteratively add the triangular faces to the mesh using the `point_lift_map` to map the faces back to 3D.

1.1.3 Triangulating the mesh

To triangulate the mesh, we use `PMP::triangulate_faces(mesh)`, this works as long as the mesh is constructed correctly and is valid. However, as a fallback we included tests to verify that both the building-to-mesh conversion and that the mesh triangulation were successful. If either of these steps failed, the volume calculation is skipped and a default value of 0.0 is given instead.

In the tile `9-284-556.city.json` there are 3 buildings of LoD 2.2 we cannot get out triangulation method to work. While most visible faces appear triangular, the Aula Conference Centre (Fig. 1c) clearly contains a hole. Consequently, we assigned these buildings a volume value of 0.0 to indicate their invalidity.



(a) Pand.0503100000018571 (b) Pand.0503100000019679 (c) Pand.0503100000019275

Figure 1: Cases that failed to be triangulated thoroughly in `9-284-556.city.json`

1.1.4 Calculating volume from mesh

Based on the previous steps and checks, we assume that for our volume calculation, the mesh is a valid solid, meaning it has a closed surface and that all its faces are properly triangulated. To calculate the volume, we defined a function called `volume_from_mesh()` which iterates over all triangular faces in the mesh, selects the first vertex in the mesh as the reference point o, and accumulates the signed volumes of the corresponding tetrahedra. Function `tetrahedron_volume()` calculates the signed volume of a single tetrahedron defined by three points (a, b, c) on a face and an additional point o, using the formula:

$$V_{\text{signed}} = \frac{1}{6}\, \vec{oa} \cdot (\vec{ab} \times \vec{ac}) \tag{1}$$

1.2 Determining each building's general orientation

The general orientation of a building is typically defined as the angle of its dominant axis, often represented by the azimuth, in degrees, clockwise from North and only between 0-90° and 270-360°. In simpler terms, its the longest edge of the minimum bounding rectangle (MBR) which encapsulates a building's 2D footprint.

To compute this dominant axis, we retrieve a building's LoD0 footprint's 3D points. Here it doesn't matter if the footprint is a valid polygon, as long as its LoD0 footprint exists and we can extract the coordinates, since in our method we only require 2D points. Since we obtain the footprint as 3D points, yet we know it's always planar and flat, we flatten them into 2D points. To compute the MBR, we use `CGAL::min_rectangle_2` which gives us the four corners of the MBR.

Since a rectangle is rectangular with its 4 vertices being 90 degrees, to find its dominant axis we only have to compare two adjacent edges (as 2D vectors). The longest of these two is then the dominant axis, and by extension, the dominant axis of the building. Note that since the orientation of the MBR as returned by `CGAL::min_rectangle_2` depends on the input order, its rotation is uncertain, thus the resulting vector could be pointing in any direction. Since our definition of the general orientation only allows for a north-facing vector, we make sure that it has a positive y-component. Lastly, using this vector, we compute the azimuth, as measured in degrees, clockwise from North, as this is the defined general orientation.

1.3 Determining all roof surfaces' areas and orientations

1.3.1 Getting the 3D ring coordinates for each 'RoofSurface'

In a CityJSON file, each CityObject contains a `geometry` array, where surfaces are defined by boundaries referencing vertex indices, along with semantic information classifying the surface type (e.g., RoofSurface, WallSurface, GroundSurface).

To extract the 3D ring coordinates for RoofSurfaces, we iterate through the `geometry` objects of each building and locate those with type `Solid`. For each geometry, we identify the surfaces classified as `RoofSurface` by checking the semantic label at `g["semantics"]["values"]`. We then retrieve the corresponding boundary ring from `g["boundaries"]`, which contains lists of vertex indices.

Each boundary may contain multiple rings: the first ring `[0]` is the outer ring, and any subsequent rings `[1], [2], ...` represent holes. We use the vertex indices in each ring to access the 3D coordinates from the global `vertices` list. For each index `v_idx`, we retrieve the corresponding coordinate triplet `[x, y, z]` to obtain the actual 3D geometry of each ring for the RoofSurface.

1.3.2 Classifying the roof orientation

To determine and classify the orientation, we first need to compute the normal for each 'RoofSurface'. To do this, assuming each outer ring contains at least three points and is planar, we compute its normal using any three points by taking the cross product of two edge vectors. Since we cannot assume the winding is counter-clockwise (CCW) which would result in an upwards facing normal (which is desired), we always check and flip the normal vector if its z-component is negative, as each roof should point upwards.

Using this normal vector, akin to the method for the building orientation, we calculate the azimuth value measured in degrees, clockwise from North. Note that we do not restrict the result; the azimuth can range from 0–360° since a roof can face any direction.

To classify the roof orientation, we first check if the surface normal's z-component is greater than 0.95, in which case we classify it as 'horizontal'. Otherwise, we compute the azimuth from the normal and assign the surface to one of eight directional classes: [NE, EN, ES, SE, SW, WS, WN, NW]. This is done by dividing the azimuth (in degrees) by 45 and taking the result modulo 8 as index of said list. We'd like to note that this an odd classification. If we were free to choose, we would use the standard 8-point compass directions (N, NE, E, etc.) or a more precise 16-point system (N, NNE, NE, ENE, E, etc.), or better yet, the azimuth value.

1.3.3 Calculating the area of each individual ring

To calculate the area of each ring, we use a triangle based area calculation. Since we compute the areas of cartesian 2D coordinates, we first use the normal vector as discussed before, to construct a local coordinate system which we use to reproject our 3D points to 2D using an orthogonal projection defined by this coordinate system which preserves the area. Then we construct a local origin point: (-1000, -1000) so it's always outside of the roof surface. Using this local origin point we compute the signed area of each individual triangle, made by $(O,\ P_i,\ P_{i+1})$, where its orientation (CCW or CW) determines if we add or subtract that area from the overall area.

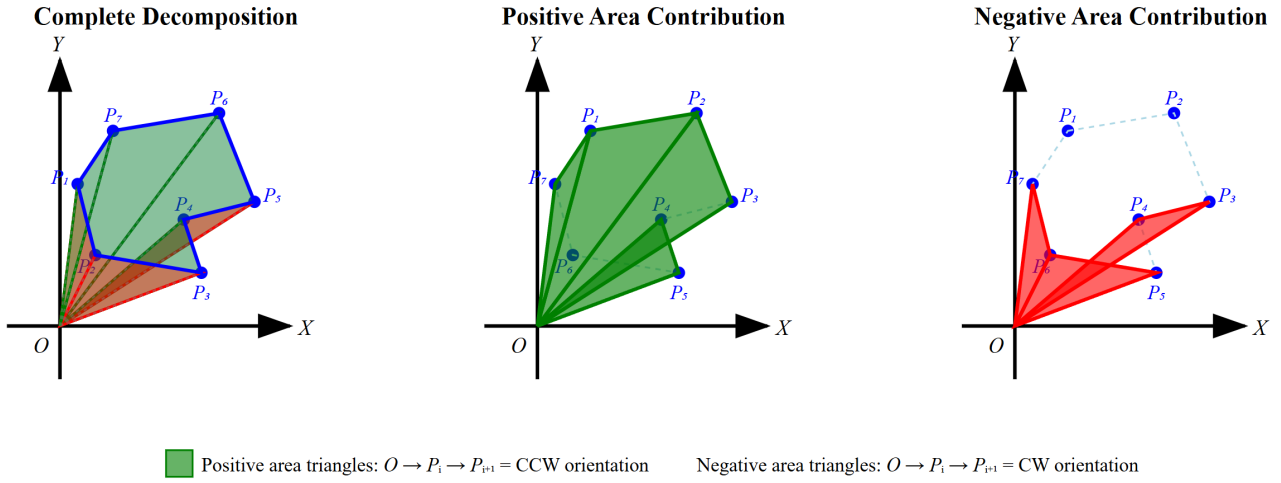


Figure 2: Illustration of the origin-based triangulation algorithm for calculating an polygon's area.

1.3.4 Calculating the area of each 'RoofSurface' surface

Let the polygon (the 'RoofSurface') have an outer ring with vertices $P_1, P_2, \ldots, P_n$, where $P_i = (x_i, y_i)$and $P_{n+1} = P_1$, and inner rings (holes) indexed by $j = 1, \ldots, m$, where each inner ring inner_j has vertices $Q_1^{(j)}, Q_2^{(j)}, \ldots, Q_{k_j}^{(j)}$. The signed area of a ring$R = \{R_1, R_2, \ldots, R_s\}$, assuming its oriented counter-clockwise with respect to an origin $O = (x_0, y_0)$ is:

$$A_{\text{ring}} = \frac{1}{2} \sum_{i=1}^{n} \big((x_i - x_0)(y_{i+1} - y_0) - (x_{i+1} - x_0)(y_i - y_0) \big), \quad \text{where} \quad (x_{n+1}, y_{n+1}) = (x_1, y_1) \tag{2}$$

As a guarantee, to account for wrongly oriented polygons, either by an invalid winding in the CityJSON, by accidental introduction in our method, we always calculate the total area of a polygon with holes as:

$$A_{\text{RoofSurface}} = |A_{\text{outer ring}}| - \sum_{j=1}^{m} \left| A_{\text{inner ring}_j} \right| \tag{3}$$

1.4 Determining each building's geometric difference between LoD1.3 and LoD2.2

To quantify the geometric difference between two different levels of detail, our case LoD1.3 and LoD2.2, we calculated the Hausdorff distance. This is a metric that measures the maximum surface deviation between two 3D models. LoD1.3 models are simple box-like geometry with flat roofs and fewer details, whilst LoD2.2 models have detailed geometry with roof features, balconies, chimneys etc.. As given in the assignment, the distance between both models cannot be directly computed, so we approximate it by sampling points on both LoD1.3 and LoD2.2 models' surfaces.

1.4.1 Triangulating Meshes

The first step for uniformly sampling points on the surface is to create a mesh of triangles, so that we can select random points on similar, simple geometry. Both LoDs were converted into watertight triangle meshes using `CGAL::Surface_mesh`. The detailed steps are described in Sections 1.1.1, 1.1.2, and 1.1.3.

1.4.2 Uniform Point Sampling

The first approach we propose is to sample 1,000 points uniformly across the surface of each mesh. To generate this uniform distribution, such to avoid bias towards smaller triangles, our sampling is area-weighted. Thus the probability of selecting a triangle becomes proportional to its area. The subsequent points within each triangle are then generated using barycentric coordinates.

1.4.3 Area-Proportional Point Sampling

However, we observed that having a set amount of points isn't ideal for buildings with deviating surface areas. Since it would mean for a twice as big building, not twice as many points would be used, and thus the spacing between the points would be bigger. Therefore we implemented a density-based sampling approach, where points counts are generated proportionally to the surface area. For each mesh, we:

- Calculate the total surface area using: $A_{triangle} = \frac{1}{2}|\vec{AB} \times \vec{AC}|$
- Use a sampling density of 25 points per m^2.
- Use `CGAL::Random_points_in_triangle_mesh_3` to generate the points.

This approach ensures a fair comparison regardless of varying building sizes. However, during implementation, we discovered that this approach presents significant scalability issues. When processing larger tile datasets containing buildings with extensive surface areas (e.g. warehouses or large commercial structures), the number of sampled points increased dramatically. For instance, a building with 10,000 m^2 of surface area would generate 250,000 sample points. Which, since our method naively compares the distances from each point to each point, would result in 250,000 $\times$ 250,000 euclidean distance calculations. To address this issue, we implemented a hybrid approach that combines our density-based sampling with a maximum point limit, in which we:

- Reduced the base density from 25 to 4 points per m^2.
- Introduced a cap of 3,000 points per building. Meaning that for buildings exceeding a 750 m^2 surface area, the sampling automatically switches from density-based to the maximum cap.

(a) LoD 1.3

(b) LoD 2.2

Figure 3: Geometric comparison between LoD 1.3 (blue points) and LoD 2.2 (red points) models. The Hausdorff distance represents the maximum deviation between these point sets.

1.4.4 Hausdorff Distance Calculation

Mathematically, it is a metric for quantifying the dissimilarity between two sets of points in 3D space. Given two finite point clouds A (representing LoD 1.3) and B (representing LoD 2.2), the Hausdorff distance $H(A, B)$ is defined as:

$$H(A, B) = \max\left(\sup_{a \in A} \inf_{b \in B} d(a, b), \sup_{b \in B} \inf_{a \in A} d(a, b)\right) \tag{4}$$

where $d(a, b)$ denotes the Euclidean distance between points a and b. This bidirectional measure captures the *worst-case* deviation between models by computing:

1. The maximum distance from any point in A to its nearest neighbor in B.
2. The maximum distance from any point in B to its nearest neighbor in A.

The final value is the larger of these two values, representing the worst-case surface deviation, with $H(A, B) = H(B, A)$ guaranteeing symmetry.

Our implementation : Computes both directed distances (LoD1.3 → LoD2.2 and LoD2.2 → LoD1.3). Uses brute-force nearest neighbor search for each sampled point and returns the maximum of both directed distances. For the dataset of `nextbk_2b.city.json` we got the value of `2.375 m` and `1.681 m` for the larger and smaller building, respectively. These value means that by how much the two different building models of LoD's differ from each other at most, at any point on the surfaces. As we can see in the Fig. 3, the LoD 2.2 buildings have more details like chimneys and dormers, with a clearly defined sloped roof instead of a flat roof in LoD 1.3. Buildings with complex roof structures showed the highest distances, with larger values indicating more significant geometric differences between the simplified and detailed representations at different Levels of Detail (LoD).

1.4.5 Validation and Visualization

To validate the sampled points we exported the points as *.ply* files (Fig. 3), with LoD 1.3 in blue and LoD 2.2 in red. For this we have used 1000 sampled points but it can be changed accordingly for more accuracy.

2 Who-did-what

- Ming-Chieh Hu (6186416): Mesh construction, Triangulation, Volume calculation, Overall program organizing, Report section 1.1.
- Neelabh Singh (6052045): Random points sampling, Hausdorff distance calculation, Point visualization, Report section 1.4.
- Daan Schlosser (5726042): General orientation calculation, Area calculation, RoofSurface orientation classification, Report section 1.2 and 1.3.